WiDS 5.0 End-Term Report

Teaching AI Agents to Walk: Reinforcement Learning with Humanoids

(UID : 19)

Chinthala Mahathi Reddy
24B1097
Mentored by Aryan Metha

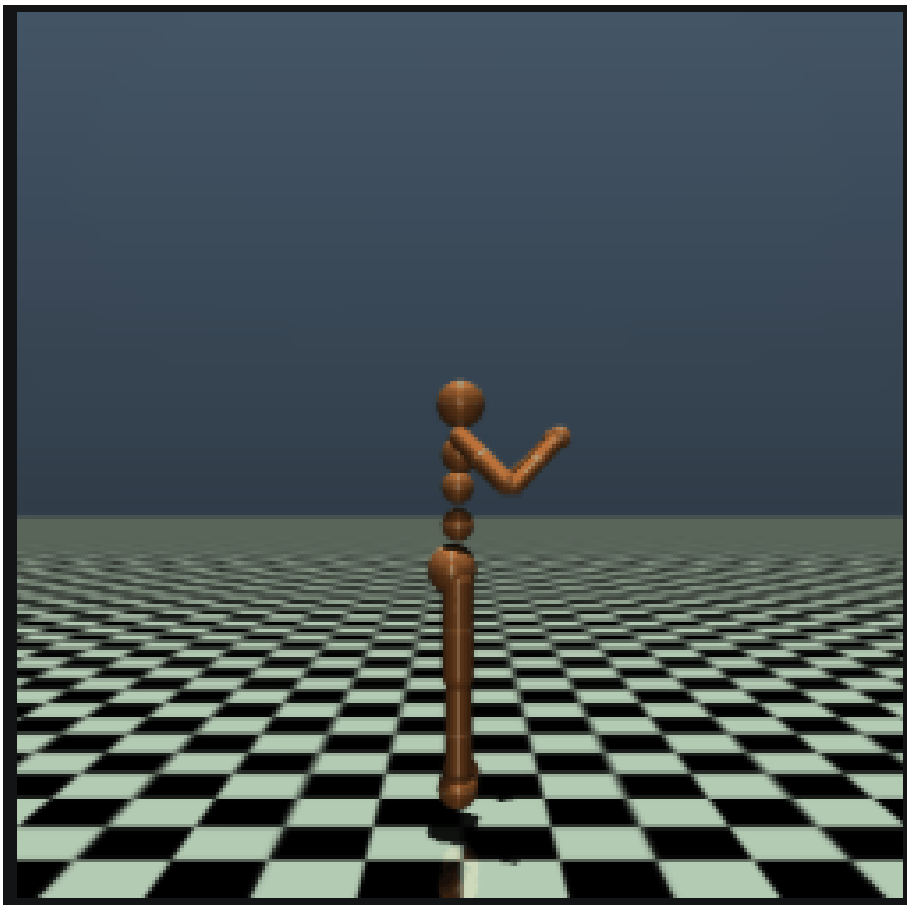

Figure 1: Reinforcement Learning with Humanoids

Contents

1 Introduction **3**

2 Week 1: Basics of Reinforcement Learning Fixed Policies **4**
2.1 Environment Design and Reward Structure 4
2.2 Fixed Policies 5

2.3 Key Learnings 5

3 Week 2: Markov Decision Processes and Dynamic Programming 6
3.1 Markov Decision Processes 6
3.2 Value Functions 6
3.3 Dynamic Programming 7
3.4 Gambler's Problem and Observations 7

4 Week 3: Model-Free Prediction, Control & Value Function Approximation 8
4.1 Model-Free Prediction Methods 8
4.1.1 Monte-Carlo Prediction 8
4.1.2 Temporal-Difference Learning 9
4.2 Model-Free Control 9
4.2.1 Monte-Carlo Control 9
4.2.2 Temporal-Difference Control: Sarsa 10
4.3 Need for Value Function Approximation 10
4.3.1 Linear Value Function Approximation 11

5 Week 4: End-Term Project — Teaching a Humanoid to Walk 12
5.1 Objective and Motivation 12
5.2 Libraries Explored 12
5.3 Environment and Algorithm 12
5.4 Training Setup 13
5.5 Results and Learnings 13

6 Conclusion 13

7 References 14

==========

1 Introduction

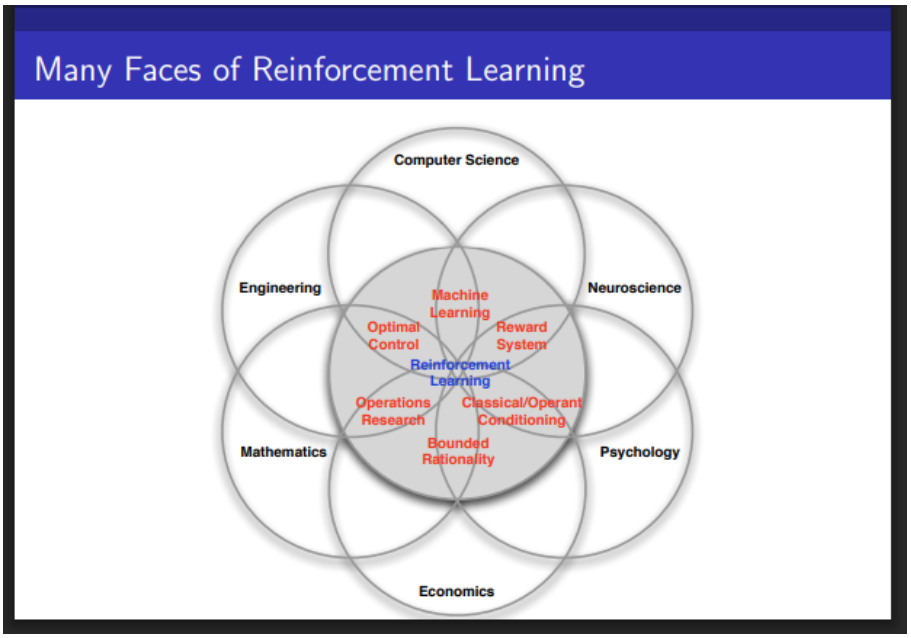

Figure 2: Reinforcement Learning

Reinforcement Learning (RL) constitutes a learning framework wherein an agent acquires the capacity to make sequential decisions through interaction with an environment, receiving feedback within the form of rewards. Distinct from supervised learning, RL does not utilize labeled input-output pairs; rather, learning is derived from trial-and-error interactions.

The WiDS 5.0 program is structured to progressively cultivate an understanding of RL, commencing with straightforward environments and static policies, progressing through Markov Decision Processes (MDPs) and Dynamic Programming, and ultimately addressing intricate continuous-control tasks, exemplified by humanoid locomotion, employing MuJoCo.

This report regards performance across Weeks 1 to 4, culminating in the end-term project: **Teaching AI Agents to Walk using Reinforcement Learning**.

2 Week 1: Basics of Reinforcement Learning Fixed Policies

2.1 Environment Design and Reward Structure

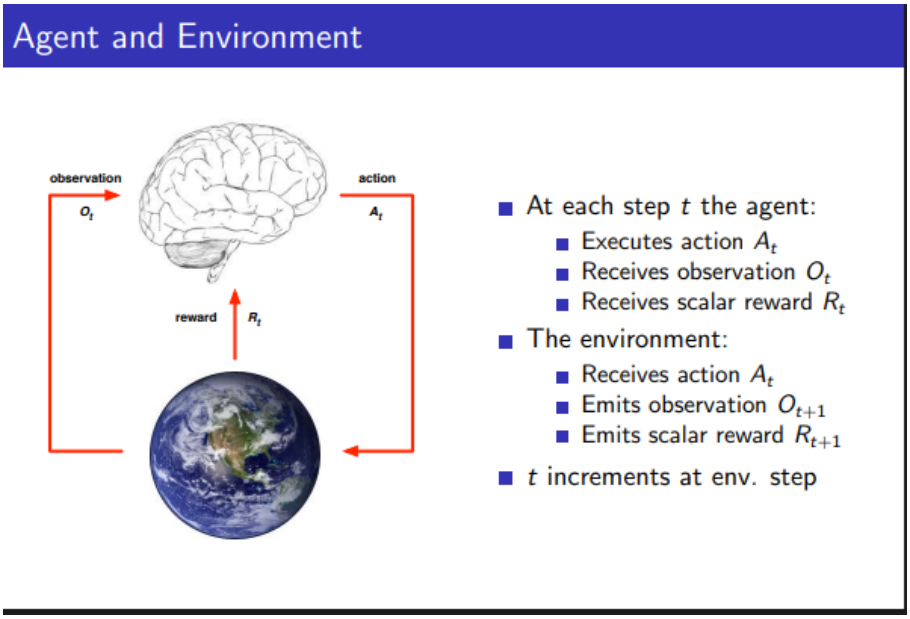

Figure 3: Agent and Environment

The first assignment focused on understanding the agent–environment interaction loop. A custom 1D GridWorld environment was implemented with the following properties:

- States: discrete positions $\{0, 1, \ldots, N-1\}$
- Actions: move left (0) or move right (1)
- Hidden goal position unknown to the agent
- Internally computed reward function

At the beginning of each episode, the agent and the goal were placed randomly at different cells.

- +10 reward for reaching the goal (terminal)
- -0.1 if the agent moves closer to the goal
- -0.2 if the agent moves away or stays at same distance
- -100 penalty if step limit is exceeded

This reward shaping demonstrated the importance of informative feedback in RL.

2.2 Fixed Policies

Three fixed (non-learning) policies were evaluated:

1. **Random Policy**: Chooses actions uniformly at random.
2. **Monotonous Policy**: Always moves in one direction.
3. **Wildcard Policy**: A heuristic policy combining direction switching and randomness.

2.3 Key Learnings

- The agent does not need to know the reward function explicitly.
- Poor policy design leads to inefficient exploration.
- Reward shaping significantly affects behavior.

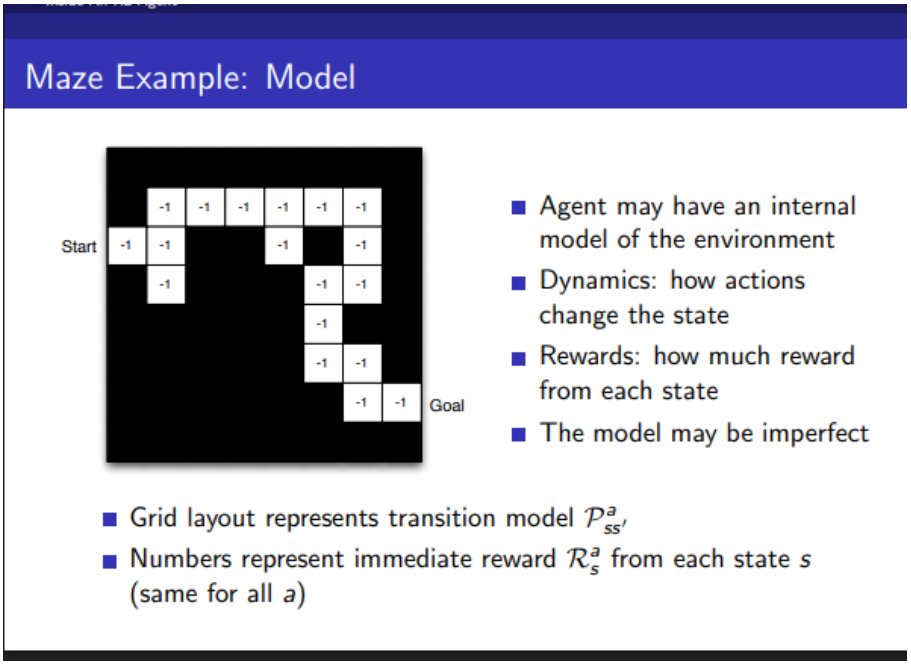

Figure 4: Maze Example

3 Week 2: Markov Decision Processes and Dynamic Programming

3.1 Markov Decision Processes

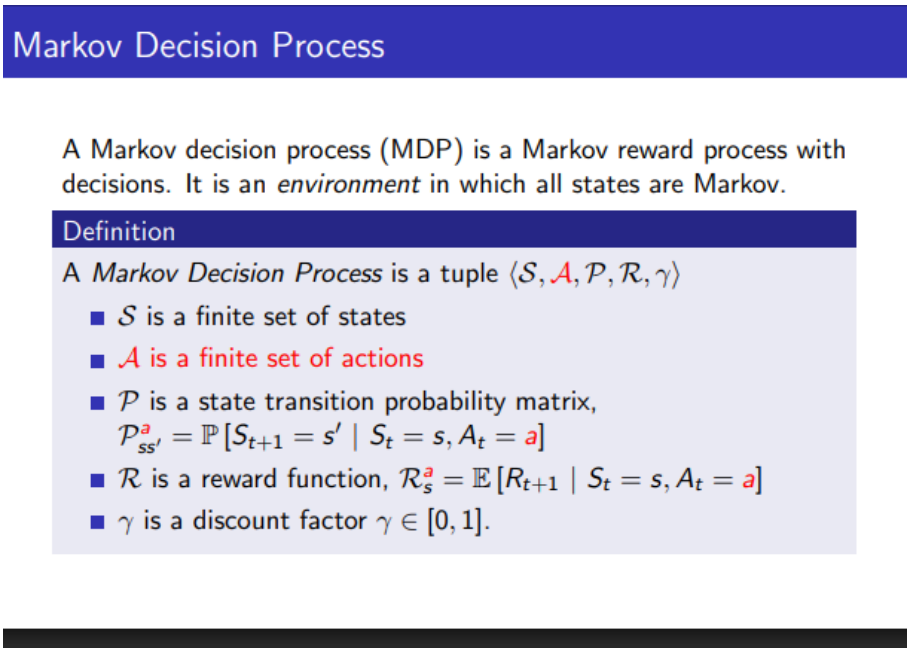

Figure 5: MDP

From lectures, an MDP is defined as:

$$\langle S, A, P, R, \gamma \rangle$$

where S is the state space, A is the action space, P the transition probabilities, R the reward function, and γ the discount factor.

MDPs provide a mathematical framework to model sequential decision-making problems.

3.2 Value Functions

- State-value function:

$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$$

- Action-value function:

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$$

3.3 Dynamic Programming

Dynamic Programming assumes full knowledge of the MDP and is used for planning. Key methods include:

- Policy Evaluation
- Policy Iteration
- Value Iteration

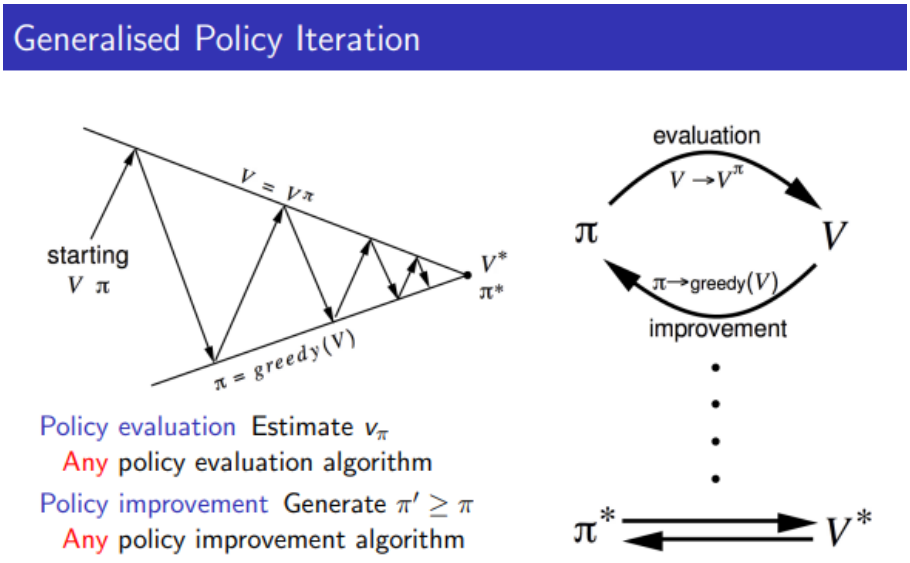

Figure 6: Policy Iteration

3.4 Gambler's Problem and Observations

Value iteration was applied to the Gambler's Problem for different probabilities $p_h = \{0.2, 0.4, 0.5, 0.7, 0.9\}$.

- Higher p_h leads to more aggressive optimal policies.
- The value function becomes steeper as winning probability increases.

4 Week 3: Model-Free Prediction, Control & Value Function Approximation

This week focused on model-free reinforcement learning methods, where the agent learns directly from sampled experience without access to the environment's transition model. The lectures covered Monte-Carlo methods, Temporal-Difference learning, model-free control algorithms, and the motivation for value function approximation in large state spaces.

4.1 Model-Free Prediction Methods

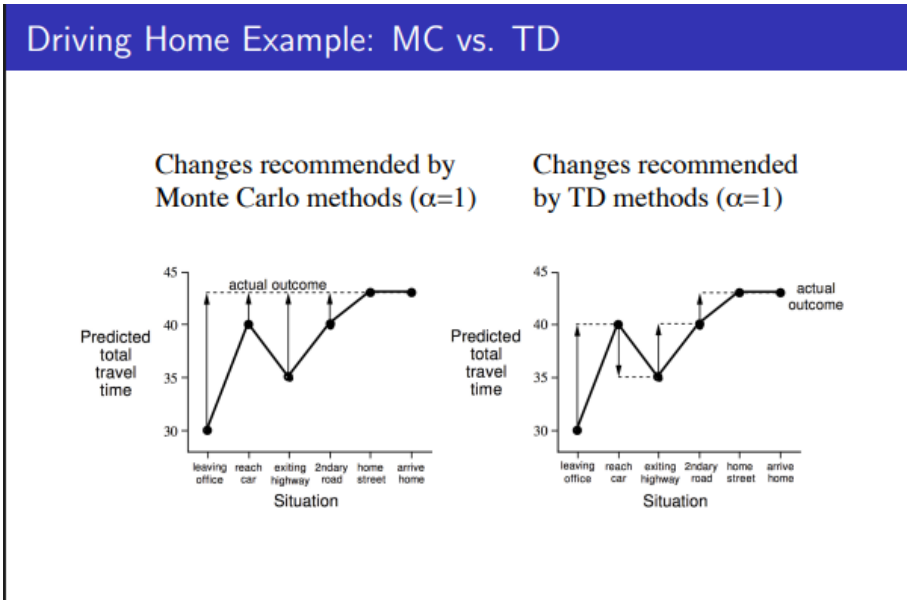

Figure 7: Monte-Carlo VS Temporal-Difference

4.1.1 Monte-Carlo Prediction

Monte Carlo (MC) methods estimate the value of a state by averaging observed returns following visits to that state. These methods:

- Learn directly from complete episodes.
- Do not bootstrap — updates depend only on actual sampled returns.
- Produce unbiased estimates.
- Suffer from high variance.

Monte Carlo prediction is simple and reliable but cannot learn until an episode terminates, which makes it unsuitable for continuing tasks.

4.1.2 Temporal-Difference Learning

Temporal-Difference (TD) learning combines ideas from Monte Carlo methods and Dynamic Programming. TD methods update value estimates using one-step lookahead targets:

$$V(S_t) \leftarrow V(S_t) + \alpha\big(R_{t+1} + \gamma V(S_{t+1}) - V(S_t)\big)$$

Key insights from TD learning include:

- TD methods bootstrap using current value estimates.
- They support online, step-by-step learning.
- They have lower variance than Monte Carlo.
- They introduce some bias due to bootstrapping.

TD learning provides faster updates and is more sample-efficient than Monte Carlo methods.

4.2 Model-Free Control

Model-free control methods extend prediction to learn optimal policies by combining policy evaluation and policy improvement using sampled experience.

4.2.1 Monte-Carlo Control

Monte Carlo control estimates the action-value function $Q(s, a)$ and improves the policy greedily with respect to these estimates.

To ensure sufficient exploration, ϵ-greedy policies are used:

- With probability $1 - \epsilon$, choose the greedy action.
- With probability ϵ, choose a random action.

The concept of *GLIE* (Greedy in the Limit with Infinite Exploration) ensures that:

- Every state–action pair is visited infinitely often.
- The policy becomes greedy in the limit.

Under GLIE conditions, Monte Carlo control converges to the optimal policy.

4.2.2 Temporal-Difference Control: Sarsa

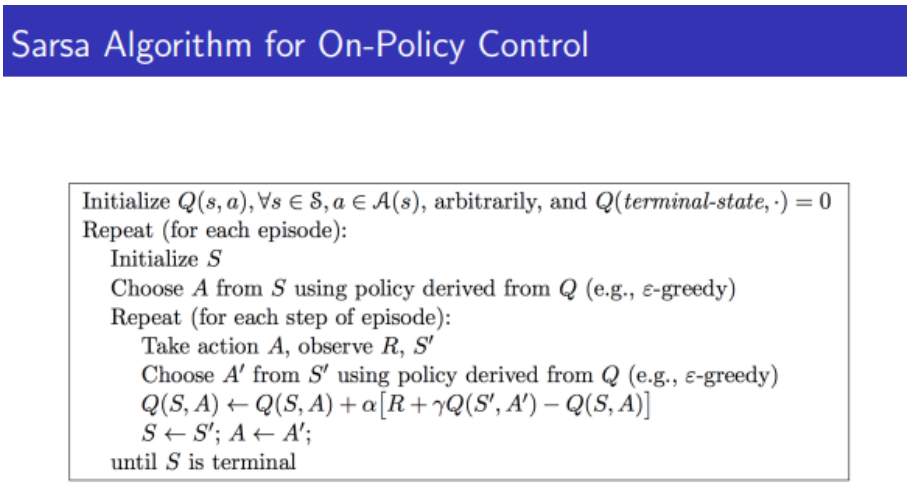

Figure 8: Sarsa Algorithm - On-policy Control

Sarsa is an on-policy TD control algorithm that updates action-values using the actual next action selected by the policy:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\big(R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\big)$$

Through Sarsa and Sarsa(λ), I learned:

- The difference between on-policy and off-policy learning.
- How bootstrapping influences convergence behavior.
- How eligibility traces help assign credit to recent state–action pairs.

These ideas were reinforced through my Easy21 experiments, where Sarsa(λ) was evaluated across different values of λ using mean squared error against the Monte-Carlo baseline.

4.3 Need for Value Function Approximation

Tabular value methods become impractical when the state or action space is large or continuous. Storing one value per state or state–action pair becomes both memory-intensive and slow to learn.

Value function approximation addresses this limitation by representing the value function using parameterized functions, allowing generalization across states.

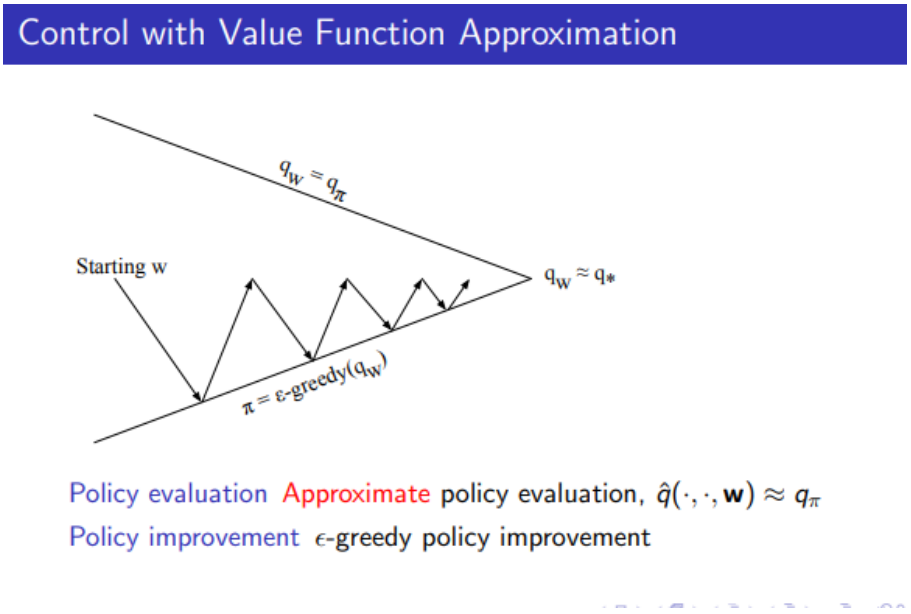

Figure 9: Value Function Approximation

4.3.1 Linear Value Function Approximation

In linear approximation, the value function is represented as:

$$\hat{v}(s, w) = x(s)^\top w$$

where:

- $x(s)$ is a feature vector describing the state
- w is a parameter vector

Key learnings include:

- Linear approximators generalize to unseen states.
- Parameters are updated using stochastic gradient descent.
- Update rule:

$$w \leftarrow w + \alpha(G_t - \hat{v}(S_t, w))x(S_t)$$

- Feature design strongly affects performance.

This idea was applied in my Easy21 assignment using coarse coding features, demonstrating how approximation enables scaling beyond tabular representations.

5 Week 4: End-Term Project — Teaching a Humanoid to Walk

5.1 Objective and Motivation

The final project applied reinforcement learning to a continuous-control task: humanoid locomotion. Unlike small tabular environments, humanoid control involves high-dimensional observations, continuous actions, and unstable physics-based dynamics.

Key challenges include:

- High-dimensional continuous state space
- Multi-joint continuous action space
- Balance and stability constraints
- Delayed reward signals

This project connected RL theory with a robotics-style control problem.

5.2 Libraries Explored

Two major RL libraries were studied and used:

Gymnasium provided the environment interface and APIs such as `reset()` and `step()`, along with observation and action space definitions. It was used to run the MuJoCo Humanoid environment.

Stable-Baselines3 was explored to understand standard implementations of modern RL algorithms (PPO, A2C, SAC), training pipelines, policy classes, and evaluation utilities. This gave insight into production-quality RL frameworks.

5.3 Environment and Algorithm

The MuJoCo Humanoid environment simulates a multi-joint robot where the goal is to move forward while maintaining balance. Observations include joint states and velocities, and actions are continuous torques.

I implemented the **REINFORCE** policy gradient algorithm, which directly optimizes policy parameters using sampled returns:

$$\nabla J(\theta) = \mathbb{E}[\nabla_\theta \log \pi_\theta(a|s)\, G_t]$$

A neural network parameterized a Gaussian policy that outputs action means and standard deviations.

5.4 Training Setup

Training was done using PyTorch and Gymnasium with:

- Two-layer neural policy network (256 units each)
- Adam optimizer, $\gamma = 0.99$
- Monte-Carlo return estimation
- No rendering during training for efficiency

Episode trajectories were collected, returns computed and normalized, and policy parameters updated using gradient ascent. The trained model was saved and later loaded for visualization.

5.5 Results and Learnings

Reward curves showed gradual improvement across episodes, indicating more stable locomotion behavior. Rendering with deterministic actions produced smoother motion.

This project strengthened my understanding of:

- Continuous-control reinforcement learning
- Policy gradient implementation
- Neural policy design
- Gymnasium environment usage
- Practical RL training and evaluation workflows

6 Conclusion

WiDS 5.0 offered a systematic exploration of reinforcement learning, commencing with fundamental principles and culminating in contemporary continuous-control applications. The curriculum began with GridWorld and Dynamic Programming, subsequently progressing through Monte-Carlo and Temporal Difference learning, model-free control strategies, and function approximation techniques.

Practical understanding was fostered through assignments like Easy21, while the humanoid project showcased the application of policy gradients and neural networks in addressing high-dimensional control challenges. Consequently, the program effectively bridged the gap between reinforcement learning theory and practical implementation, utilizing Gymnasium, Stable-Baselines3, and PyTorch.

7 References

1. https://www.geeksforgeeks.org/machine-learning/introduction-machine-learning/
2. https://www.youtube.com/playlist?list=PLqYmG7hTraZDM-OYHWgPebj2MfCFzFObQ&authuser=0
3. https://drive.google.com/file/d/1z9YOw4FXeR155227yUUMc7_Hxv3POE_8/view?usp=classroom_web&authuser=0
4. https://stable-baselines3.readthedocs.io/en/master/?authuser=0
5. http://gymnasium.farama.org/?authuser=0